

LECTURE 1: INTRODUCTION – A UNIFIED FRAMEWORK FOR LARGE-SCALE BLOCK-STRUCTURED OPTIMIZATION

Mingyi Hong

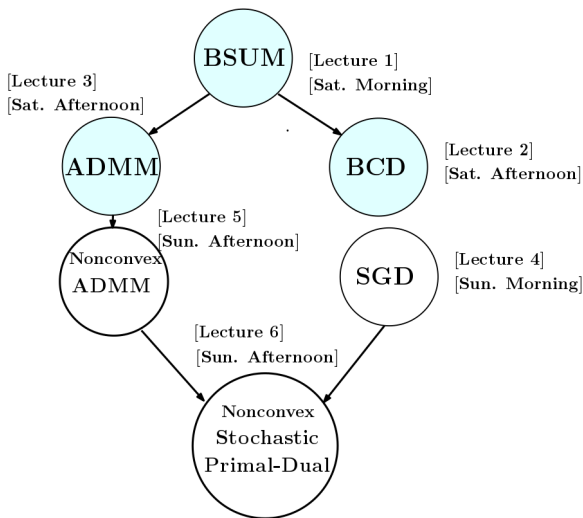
Iowa State University
IMSE and ECpE Department

June 2016, @ MOA Workshop

Main references (tutorials)

- M. Hong, M. Razaviyayn, Z.-Q. Luo, and J.-S. Pang, “A unified algorithmic framework for block structured optimization involving big data,” IEEE Signal Processing Magazine, vol. 33, no. 1, pp. 57 - 77, 2016.
- M. Razaviyayn, M. Hong, and Z.-Q. Luo, “A unified convergence analysis of block successive minimization methods for nonsmooth optimization,” SIAM Journal on Optimization, vol. 23, no. 2, pp. 1126 - 1153, 2013.

Schedule (temporary)



This Talk

- **Main Theme.**

Introduce a family of efficient algorithms for large-scale convex/nonconvex optimization

Offer new perspectives to a few old/new algorithms

Discuss applications in ML, Networking, SP, etc

- Based on a recent survey paper [Razaviyayn-H.-Luo-Pang 16]

- **Take Home Message.** A useful tool box for designing and analyzing optimization algorithms

“A unified algorithmic framework for block-structured optimization involving big data”,
Signal Processing Magazine, 2016.

Outline

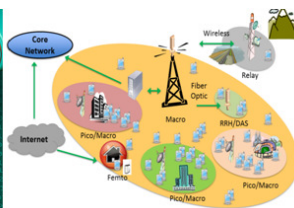
- 1 Introduction: Block Structured Problems
- 2 The BSUM Framework
- 3 Properties of BSUM
- 4 Algorithms Covered By BSUM
- 5 Applications of BSUM in SP, ML, COM
- 6 Extension
- 7 Conclusion

Outline

- 1 Introduction: Block Structured Problems
- 2 The BSUM Framework
- 3 Properties of BSUM
- 4 Algorithms Covered By BSUM
- 5 Applications of BSUM in SP, ML, COM
- 6 Extension
- 7 Conclusion

Introduction

- Advances in sensor, communication and computational technologies – easy data acquisition
- **Big data sets** in many areas in engineering, biological, social physical sciences
- Emerging problems in image processing, network science, machine learning easily exceed **millions/billions** of variables
- A fundamental tool: large-scale optimization and computation



Large-Scale Optimization

- Generally, optimization algorithms for solving problems of such huge sizes should satisfy
 - **Simple Subproblem:** Each of their computational steps must be **simple and easy**
 - **Parallel Implementable:** Algorithm is implementable in **distributed and/or parallel** manner (e.g., in HPC clusters)
 - **Fast Convergence:** A high-quality solution can be found using a **small number of iterations**
- **Key:** Explore intrinsic decomposability of the problem

The BCD Algorithm

- A popular family of optimization algorithms: **block coordinate descent** (BCD) method (a.k.a. the alternating minimization)
- The basic steps:
 - **Partition** the entire optimization variables into small blocks
 - **Optimize** one block variable (or few blocks of variables) at each iteration, while holding the remaining variables fixed.
- Graphically...

The BCD Algorithm

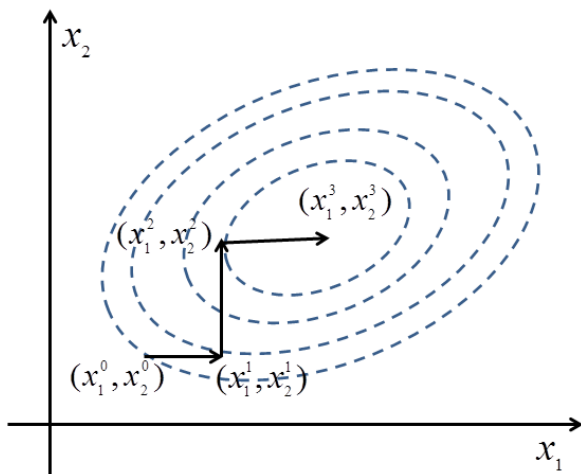


Figure: Illustration of the BCD method for a two-dimensional problem. The dashed curves represent the contour of the objective function, the solid lines represent the progress of the algorithm.

Introduction

- Consider the following structured optimization problem

$$\begin{array}{ll} \min & f(x_1, \dots, x_n) \\ \text{s.t.} & x_i \in \mathcal{X}_i, \forall i \end{array}$$

$f : \mathbb{R}^n \rightarrow \mathbb{R}$ continuous function; X_i closed convex set

- Block Coordinate Descent (BCD) method: choose i

$$\begin{array}{ll} x_i^{r+1} &= \arg \min_{x_i \in \mathcal{X}_i} f(x_1^r, \dots, x_{i-1}^r, x_i, x_{i+1}^r, \dots, x_n^r) \\ x_k^{r+1} &= x_k^r, \quad \forall k \neq i \end{array}$$

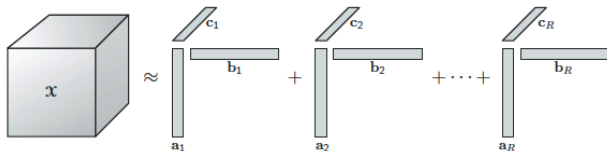
- Choice of i : **cyclic** v.s. **Gauss-Southwell** (max improvement)
- Popular due to its simplicity and scalability

Example: tensor decomposition

- Tensor $\mathfrak{X} \in \mathbb{R}^{I \times J \times K}$
- CANDECOMP/PARAFAC decomposition:

$$\mathfrak{X} = \sum_{\ell=1}^L a_{\ell} \circ b_{\ell} \circ c_{\ell}$$

- a_{ℓ} : ℓ -th column of A ; similarly define b_{ℓ}, c_{ℓ}
- $A \in \mathbb{R}^{I \times L}$, $B \in \mathbb{R}^{J \times L}$, and $C \in \mathbb{R}^{K \times L}$



Example: tensor decomposition

- Alternating Least Squares (ALS); see [Carroll *et al.* 70, Harshman 70]

$$\min_{A,B,C} \left\| \mathfrak{X} - \sum_{\ell=1}^L a_{\ell} \circ b_{\ell} \circ c_{\ell} \right\|^2$$

- Use BCD, closed form updates
- Each step solves a least squares problem

Example: Dictionary Learning for Sparse Representation

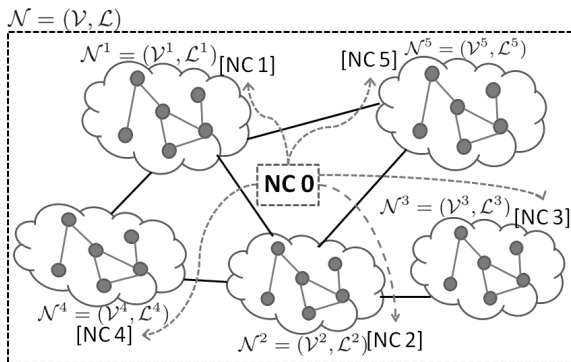
- In many ML problems, need to “reconstruct” the data
- **Assumption**: observation $\{\mathbf{b}_i\}$ is a **sparse** linear combination of **dictionary items**

$$\min_{\mathbf{D}, \{\mathbf{x}_i\}} \frac{1}{2} \sum_{i=1}^N \|\mathbf{D}\mathbf{x}_i - \mathbf{b}_i\|^2 + h(\mathbf{x}_i)$$

- N : # of data points
- $\mathbf{x}_i \in \mathbb{R}^{B \times 1}$: coefficient for the observation $\mathbf{b}_i$
- $\mathbf{D} \in \mathbb{R}^{K \times B}$: A **dictionary** with B “dictionary items”
- $h(\cdot)$: nonsmooth regularizers that promote sparsity
- A natural block-structured problem; can use BCD

Example: Multi-Commodity Routing Problem (TE)

- Consider a **multi-commodity routing problem**; arises in the design of next-G cloud-based communication networks
- A network $\mathcal{N} = (\mathcal{V}, \mathcal{L})$ controlled by $K + 1$ NCs
- Objective:** Transport M data flows over the network



Example: Multi-Commodity Routing Problem (TE)

- **Link capacity constraints.** The flow rate on link ℓ for m th commodity is denoted as $f_{\ell,m}$; should satisfy

$$\sum_{m=1}^M f_{\ell,m} \leq C_{\ell}, \quad \forall \ell \in \mathcal{L}, \quad (1.1)$$

- **Flow conservation constraints.** For any node $v \in \mathcal{V}$ and data flow m , **total incoming flow = total outgoing flow**:

$$\sum_{l \in \text{In}(v)} f_{l,m} + 1_{v=s(m)} r_m = \sum_{l \in \text{Out}(v)} f_{l,m} + 1_{v=d(m)} r_m \quad (1.2)$$

- Max-min fairness problem (**a linear program**)

$$\max_{\mathbf{f}, \mathbf{r}} \quad r_{\min} \quad \text{s.t.} \quad \mathbf{f} \geq 0, \quad r_m \geq r_{\min}, \quad m = 1, \dots, M \quad (1.3a)$$

$$(1.1) \text{ and } (1.2), \quad (1.3b)$$

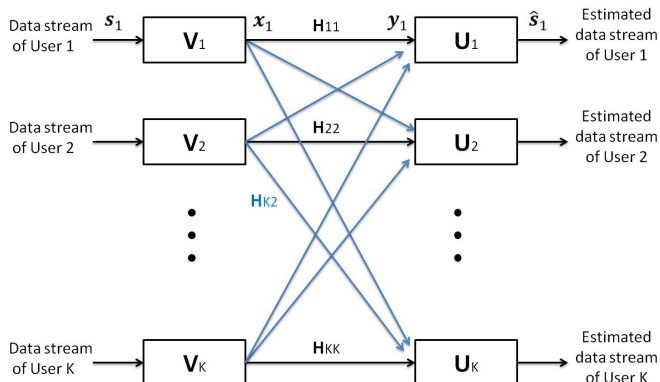
Example: Multi-Commodity Routing Problem (TE)

- Problem naturally has the block structures
- Each NC i controls a collection of local flow variables

$$f_{\ell,m}, \ell \in \mathcal{L}_i$$

- Use BCD?

Example: Precoder design in MIMO systems



- K Tx-Rx pairs, $\mathbf{H}_{kj} \in \mathbb{C}^{M \times N}$, M/N : # of antennas at each Tx/Rx
- $\mathbf{y}_k = \mathbf{H}_{kk}\mathbf{x}_k + \sum_{j \neq k} \mathbf{H}_{kj}\mathbf{x}_j + \mathbf{n}_k$, $\mathbf{x}_k = \mathbf{V}_k \mathbf{s}_k$, $\hat{\mathbf{s}}_k = \mathbf{U}_k^H \mathbf{y}_k$

Example: Precoder design in MIMO systems

- Transmit covariance matrix $\mathbf{Q}_k = \mathbf{V}_k \mathbf{V}_k^T$
- Optimization problem maximizing the throughput

$$\begin{aligned} \max_{\mathbf{Q}} \quad & \sum_{k=1}^K \log \det \left(\mathbf{I} + \underbrace{\mathbf{H}_{kk} \mathbf{Q}_k \mathbf{H}_{kk}^H}_{\text{useful signal}} \underbrace{(\sigma_k^2 + \sum_{j \neq k} \mathbf{H}_{kj} \mathbf{Q}_j \mathbf{H}_{kj}^H)^{-1}}_{\text{interference}} \right) \\ \text{s.t.} \quad & \underbrace{\text{Tr}(\mathbf{Q}_k) \leq P_k}_{\text{per-user power budget constraint}}, \mathbf{Q}_k \succeq 0, \forall k \end{aligned}$$

- A block structured problem, with the blocks being $\{\mathbf{Q}_k\}_{k=1}^K$
- Applying BCD – optimizing one user at a time (easy subproblem?)

Optimal resource allocation: Special Case

- In a MAC channel with a single receiver, the previous problem can be simplified to

$$\begin{aligned} \max_{\mathbf{Q}} \quad & \log \det \left(\mathbf{I} + \sum_k \mathbf{H}_{kk} \mathbf{Q}_k \mathbf{H}_{kk}^H \right) \\ \text{s.t.} \quad & \text{Tr}(\mathbf{Q}_k) \leq P_k, \mathbf{Q}_k \succeq 0, \forall k \end{aligned}$$

- Global optimal solution can be found by the Iterative Water-Filling Algorithm (IWFA) [Yu *et al* 03]
- Simply the classical Cyclic BCD

Convergence of BCD

- “Nonlinear Programming”, D.P. Bertsekas:

Proposition 2.7.1: (Convergence of Block Coordinate Descent) Suppose that f is continuously differentiable over the set X of Eq. (2.111). Furthermore, suppose that for each $x = (x_1, \dots, x_m) \in X$ and i ,

$$f(x_1, \dots, x_{i-1}, \xi, x_{i+1}, \dots, x_m)$$

viewed as a function of ξ , attains a unique minimum $\bar{\xi}$ over X_i , and is monotonically nonincreasing in the interval from x_i to $\bar{\xi}$. Let $\{x^k\}$ be the sequence generated by the block coordinate descent method (2.112). Then, every limit point of $\{x^k\}$ is a stationary point.

- Smoothness** assumption (relaxed later in [Tseng 01])
- Uniqueness** of minimizer (necessary, see [Powell 73] for a counter example)
- Subproblem **easily solvable**

Limit of BCD

- BCD becomes quite limited for solving large-scale problems
- What if the optimizer is **not unique** (the tensor decomposition problem)?
- What if the sub-problem is **not easy** e.g. non-convex, or convex but not closed form, or is very expensive to obtain an exact global min (the precoder design problem)?
- Coupling constraints (the multi-commodity TE problem)?
- How to enable parallelization?
- Any theoretical guarantees?

The Counter-Example

Consider the following unconstrained problem

$$\begin{aligned} \min \quad f(x) := & -x_1x_2 - x_2x_3 - x_3x_1 + (x_1 - 1)_+^2 + (-x_1 - 1)_+^2 \\ & + (x_2 - 1)_+^2 + (-x_2 - 1)_+^2 + (x_3 - 1)_+^2 + (-x_3 - 1)_+^2 \end{aligned}$$

where the notation $(z)_+^2$ means $(\max\{0, z\})^2$. In this case, fixing (x_2, x_3) and optimizing over x_1 yields the following solution

$$x_1 = \begin{cases} (1 + \frac{1}{2}|x_2 + x_3|)\text{sign}(x_2 + x_3), & \text{if } x_2 + x_3 \neq 0, \\ [-1, 1], & \text{otherwise} \end{cases}. \quad (1.4)$$

Similar solution can be obtained for x_2 and x_3 as well. Let $(x_1^0, x_2^0, x_3^0) = (-1 - \epsilon, 1 + \frac{1}{2}\epsilon, -1 - \frac{1}{4}\epsilon)$ for some $\epsilon > 0$. (1.4).

The Counter-Example

- It can be shown that applying the cyclic BCD, the iterates will be **cycling** around six points $(1, 1, -1)$, $(1, -1, -1)$, $(1, -1, 1)$, $(-1, -1, 1)$, $(-1, 1, 1)$, $(-1, 1, -1)$, and **none of these six** points is a stationary solution of the original problem.
- **The reason:** In any one of the six limit points above, there are at least **two subproblems** that have multiple optimal solutions.
- For example, at $(1, 1, -1)$ and fixing x_2, x_3 (resp. x_1, x_3), the optimal solution for x_1 (resp. x_2) is any element in the interval $[-1, 1]$; cf. (1.4).

Practical solution

- Approximate $f(x_1^r, \dots, x_{i-1}^r, x_i, x_{i+1}^r, \dots, x_n^r)$ with $u_i(x_i, x^r)$

$$x_i^{r+1} = \arg \min_{x_i \in \mathcal{X}_i} u_i(x_i, x^r)$$

- Very popular in various applications (many examples to come)
- Key Questions:** How to pick the approximation? General guidelines? Performance?

Outline

- 1 Introduction: Block Structured Problems
- 2 The BSUM Framework**
- 3 Properties of BSUM
- 4 Algorithms Covered By BSUM
- 5 Applications of BSUM in SP, ML, COM
- 6 Extension
- 7 Conclusion

The BSUM Method

- Introduce a unifying framework – the **Block Successive Upper Bound Minimization** (BSUM) [Razaviyayn-H.-Luo 13]
- Generalizes all BCD type algorithms
- **Main Idea**: Successively optimize certain **upper-bounds** of the original objective, in a **block-by-block manner**
- Significantly expands the application domain of BCD
- Covers many classical algorithms
 - Expectation Maximization (EM) [Dempster et al 77]
 - Concave-Convex Procedure (CCCP) [Yuille 03]
 - multiplicative Nonnegative Matrix Factorization (NMF) [Lee and Seung 99]

The BSUM Algorithm

- 1 Find a feasible point $x^0 \in X$ and set $r = 0$
- 2 **repeat**
- 3 Pick index set $\mathcal{I}^r$
- 3 Let $x_i^r \in \arg \min_{x_i \in \mathcal{X}_i} u_i(x_i, x^{r-1}), \forall i \in \mathcal{I}^r$
- 4 Set $x_k^r = x_k^{r-1}, \forall k \notin \mathcal{I}^r$
- 5 $r = r + 1,$
- 6 **until** some convergence criterion is met

Table: Pseudo code of the BSUM algorithm

The BSUM Algorithm

- $\mathcal{I}^r$ is an index set determining which variable to pick; potentially multiple variables can be picked
- $\mathcal{I}^r = \{(r \bmod n) + 1\}$ is the classical cyclic selection rule
- If no approximation is used, BSUM reduces to BCD
- Graphically...

Graphical Illustration of BSUM

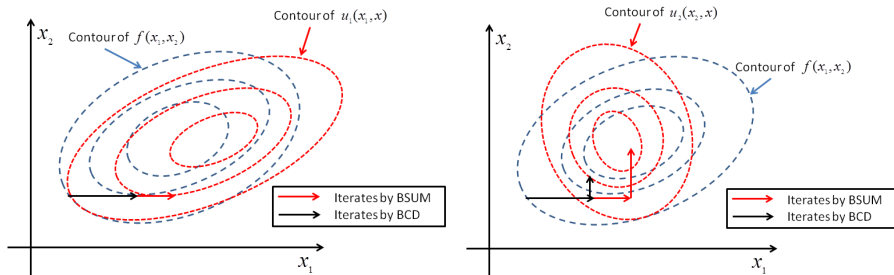


Figure: Comparison of BSUM and BCD methods for solving a two-dimensional problem. The blue dashed curves represent the contour of the objective function $f(x_1, x_2)$, the red dotted curves represent the contours of the upper-bound functions $u_1(x_1, x)$ and $u_2(x_2, x)$;

Outline

- 1 Introduction: Block Structured Problems
- 2 The BSUM Framework
- 3 Properties of BSUM**
- 4 Algorithms Covered By BSUM
- 5 Applications of BSUM in SP, ML, COM
- 6 Extension
- 7 Conclusion

What is an “appropriate” approximation?

- The approximation should be tight: $u_i(x_i^r, x^r) = f(x^r)$
- Locally should behave the same

$$u'_i(\mathbf{x}_i, x^r; d_i) \Big|_{x_i=x_i^r} = f'(x^r; d), \quad \forall d = (0, \dots, d_i, \dots, 0)$$

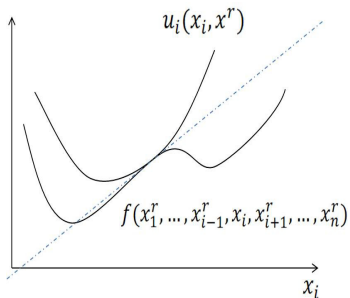
- Approximation should be an **upper-bound** (Why?)

$$u_i(\mathbf{x}_i, x^r) \geq f(x_1^r, \dots, x_{i-1}^r, \mathbf{x}_i, x_{i+1}^r, \dots, x_n^r)$$

- Continuous approximation (predictability of the approximation)

$$u_i(x_i, y) \text{ should be continuous in } (x_i, y)$$

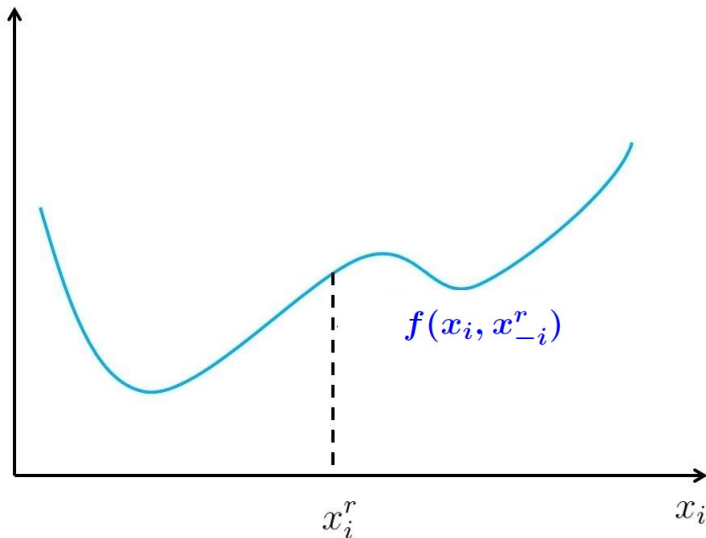
Assumptions for BSUM



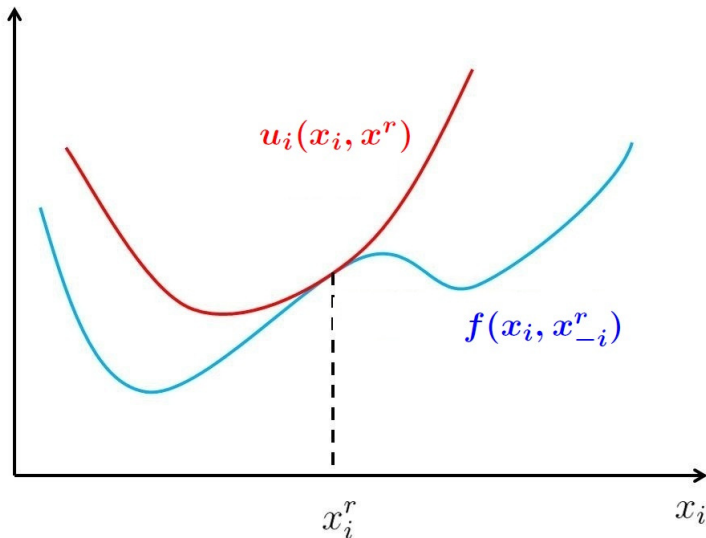
- $u_i(x_i^r, x^r) = f(x^r)$
- $u_i'(x_i, x^r; d_i) \Big|_{x_i=x_i^r} = f'(x^r; d)$
- $u_i(x_i, x^r) \geq f(x_1^r, \dots, x_{i-1}^r, x_i, x_{i+1}^r, \dots, x_n^r)$
- $u_i(x_i, y)$ is continuous in (x_i, y)

- “BSUM Assumptions”

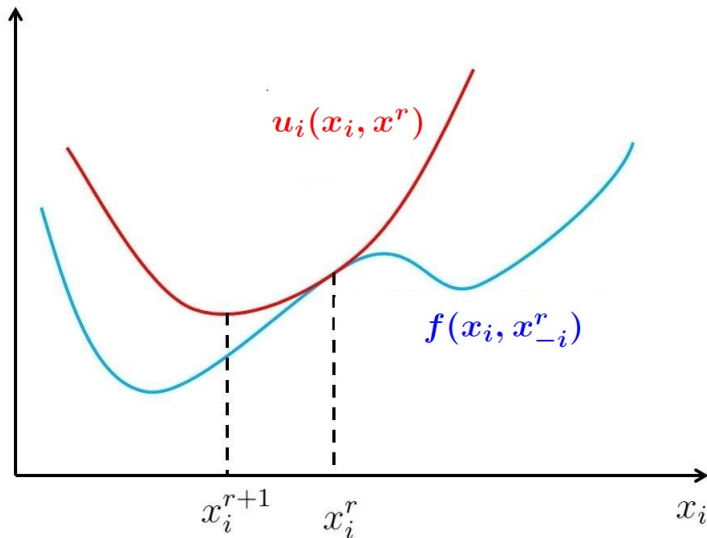
Idea of BSUM



Idea of BSUM



Idea of BSUM



Popular Upper-bounds

- **Proximal Upper Bound:** Given a constant $\gamma > 0$, construct a bound by adding a quadratic penalization

$$u_i(x_i, z) := f(x_i, z_{-i}) + \frac{\gamma}{2} \|x_i - z_i\|^2.$$

- **Quadratic Upper Bound:** Suppose

$$f(x) = g(x_1, \dots, x_n) + h(x_1, \dots, x_n),$$

where g is smooth with $\mathbf{H}_i$ as the Hessian matrix for the i th block. Then construct the following bound

$$u_i(x_i, z) := g(z_i, z_{-i}) + \langle \nabla_i g(z_i, z_{-i}), x_i - z_i \rangle + \frac{1}{2} (x_i - z_i)^T \Phi_i (x_i - z_i) + h(x_i, z_{-i}).$$

where $\Phi_i \succeq \mathbf{H}_i$ is a positive semidefinite matrix.

Popular Upper-bounds (cont.)

- **Linear Upper Bound:** Suppose f is differentiable and concave, then one can construct

$$u_i(x_i, z) := f(z_i, z_{-i}) + \langle \nabla_i f(z_i, z_{-i}), x_i - z_i \rangle.$$

- **Jensen's Upper Bound:** Suppose $f(x) := f(a_1^T x_1, \dots, a_n^T x_n)$ where $a_i \in \mathbb{R}^{m_i}$ is a coefficient vector, and f is convex with respect to each $a_i^T x_i$. Let $w_i \in \mathbb{R}_+^{m_i}$ denote a weight vector with $\|w_i\|_1 = 1$. Then one can use **Jensen's inequality** and construct

$$u_i(x_i, z) := \sum_{j=1}^{m_i} w_i(j) f\left(\frac{a_i(j)}{w_i(j)} (x_i(j) - z_i(j)) + a_i^T z_i, z_{-i}\right)$$

where $w_i(j)$ represents the j -th element in vector w_i .

Convergence Analysis

- Next we begin to analyze the convergence of the BSUM
- There are a few assumptions/conditions that we need to make
- We will illustrate them one by one

Optimality Condition

- Stationary solutions: The point x^* is a *stationary solution* if

$$f'(x^*; d) \geq 0, \forall d \text{ such that } x + d \in \mathcal{X}.$$

Let $\mathcal{X}^*$ denote the set of stationary solutions.

- Coordinatewise minimum solutions: $\hat{x} \in \mathcal{X}$ is coordinatewise minimum of with respect to the coordinates in $\mathbb{R}^{m_1}, \mathbb{R}^{m_2}, \dots, \mathbb{R}^{m_n}$, if

$$f(\hat{x} + d_k^0) \geq f(\hat{x}), \quad \forall d_k \in \mathbb{R}^{m_k} \quad \text{with} \quad \hat{x} + d_k^0 \in \mathcal{X}, \quad \forall k$$

where $d_k^0 = (0, \dots, d_k, \dots, 0)$

Separable Constraints

- Here we need $x_i \in X_i$ for all i
- And that the X_i 's are **separable**
- What do I mean?

Separable Constraints

Consider the following simple quadratic problem:

$$\min \quad x_1^2 + x_2^2, \quad \text{s.t.} \quad x_1 + x_2 = 2.$$

- The optimal objective value is 2.
- Consider the BSUM algorithm with an arbitrary approximation function satisfying Assumption A, but initiated at the point $(x_1^0, x_2^0) = (0, 2)$
- Then the BCD/BSUM method will be stuck at this non-interesting point without making any progress
- Because it is not possible to change a single block without violating the coupling constraint.

Separable Constraints

Consider the following simple quadratic problem:

$$\min \quad x_1^2 + x_2^2, \quad \text{s.t.} \quad x_1 + x_2 = 2.$$

- The optimal objective value is 2.
- Consider the BSUM algorithm with an arbitrary approximation function satisfying Assumption A, but initiated at the point $(x_1^0, x_2^0) = (0, 2)$
- Then the BCD/BSUM method will be stuck at this non-interesting point without making any progress
- Because it is not possible to change a single block without violating the coupling constraint.

Separable Constraints

Consider the following simple quadratic problem:

$$\min \quad x_1^2 + x_2^2, \quad \text{s.t.} \quad x_1 + x_2 = 2.$$

- The optimal objective value is 2.
- Consider the BSUM algorithm with an arbitrary approximation function satisfying Assumption A, but initiated at the point $(x_1^0, x_2^0) = (0, 2)$
- Then the BCD/BSUM method will be stuck at this non-interesting point without making any progress
- Because it is not possible to change a single block without violating the coupling constraint.

Separable Constraints

Consider the following simple quadratic problem:

$$\min \quad x_1^2 + x_2^2, \quad \text{s.t.} \quad x_1 + x_2 = 2.$$

- The optimal objective value is 2.
- Consider the BSUM algorithm with an arbitrary approximation function satisfying Assumption A, but initiated at the point $(x_1^0, x_2^0) = (0, 2)$
- Then the BCD/BSUM method will be stuck at this non-interesting point without making any progress
- Because it is not possible to change a single block without violating the coupling constraint.

Separable Constraints

Consider the following simple quadratic problem:

$$\min x_1^2 + x_2^2, \quad \text{s.t.} \quad x_1 + x_2 = 2.$$

- The optimal objective value is 2.
- Consider the BSUM algorithm with an arbitrary approximation function satisfying Assumption A, but initiated at the point $(x_1^0, x_2^0) = (0, 2)$
- Then the BCD/BSUM method will be stuck at this non-interesting point without making any progress
- Because it is not possible to change a single block without violating the coupling constraint.

The Regularity

- A given point $x \in X$ is called **regular** if
coordinate minimum solution = stationary solution
- What do I mean?...

- **Stationary solutions:** The point x^* is a *stationary solution* if $f'(x^*; d) \geq 0$ for all d such that $x + d \in X$.
- **Coordinatewise minimum solutions:** $\hat{x} \in X$ is coordinatewise minimum with respect to the coordinates in $\mathbb{R}^{m_1}, \mathbb{R}^{m_2}, \dots, \mathbb{R}^{m_n}$, if

$$f(\hat{x} + d_k^0) \geq f(\hat{x}), \quad \forall d_k \in \mathbb{R}^{m_k} \quad \text{with} \quad \hat{x} + d_k^0 \in X, \quad \forall k = 1, 2, \dots, n,$$

where $d_k^0 = (0, \dots, d_k, \dots, 0)$.

Table: Optimality Conditions.

Illustration: The Regularity Condition

- Consider the convex function $f(z) = \|Az\|_1$, where $A = \begin{bmatrix} 3 & 4 \\ 2 & 1 \end{bmatrix} \in \mathbb{R}^{2 \times 2}$
- Unique stationary solution $(x_1, x_2) = (0, 0)$ (global optimal)
- However, the point $\hat{z} = (-4, 3)$ is a coordinatewise minimum, but not a stationary

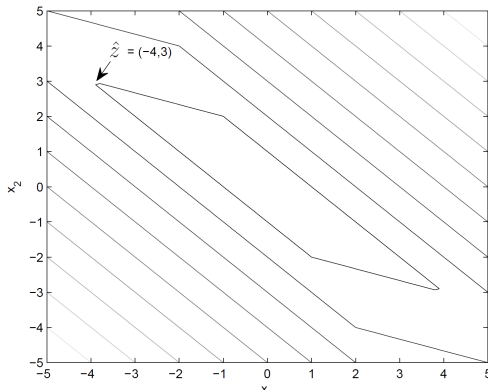


Illustration: The Regularity Condition

- Consider the convex function $f(z) = \|Az\|_1$, where $A = \begin{bmatrix} 3 & 4 \\ 2 & 1 \end{bmatrix} \in \mathbb{R}^{2 \times 2}$
- Unique **stationary solution** $(x_1, x_2) = (0, 0)$ (global optimal)
- However, the point $\hat{z} = (-4, 3)$ is a **coordinatewise minimum**, but **not** a stationary

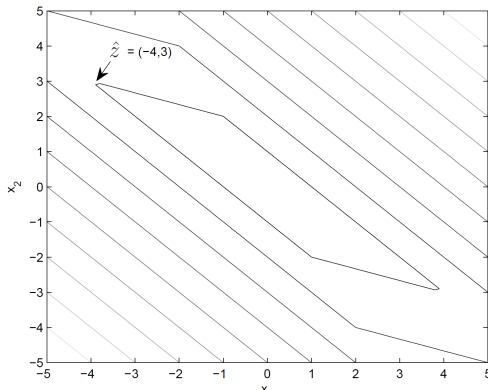
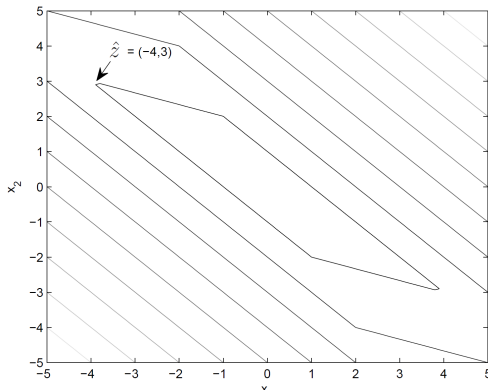


Illustration: The Regularity Condition

- Consider the convex function $f(z) = \|Az\|_1$, where $A = \begin{bmatrix} 3 & 4 \\ 2 & 1 \end{bmatrix} \in \mathbb{R}^{2 \times 2}$
- Unique **stationary solution** $(x_1, x_2) = (0, 0)$ (global optimal)
- However, the point $\hat{z} = (-4, 3)$ is a **coordinatewise minimum**, but **not** a stationary



The Regularity Condition

- In practice, special attention is needed to detect regularity
- **Good News:** For a large and popular class of problem where the nonsmooth function is *separable across the blocks*, all feasible points $x \in X$ are regular.

$$\min f(x) := g(x_1, \dots, x_n) + \underbrace{\sum_{i=1}^n h_i(x_i)}_{\text{separable nonsmooth functions}}, \quad \text{s.t. } x_i \in X_i, \forall i$$

where $g(\cdot)$ is smooth but possible non-convex functions; each h_i is a convex but non-smooth function.

The Regularity Condition

- In practice, special attention is needed to detect regularity
- **Good News:** For a large and popular class of problem where the nonsmooth function is *separable across the blocks*, all feasible points $x \in X$ are regular.

$$\min f(x) := g(x_1, \dots, x_n) + \underbrace{\sum_{i=1}^n h_i(x_i)}_{\text{separable nonsmooth functions}}, \quad \text{s.t. } x_i \in X_i, \forall i$$

where $g(\cdot)$ is smooth but possible non-convex functions; each h_i is a convex but non-smooth function.

Main Convergence Analysis

- Below we present one version of the convergence analysis
- Many variants – different assumption on choices of $\mathcal{I}^r$, on the boundedness of feasible set, etc.

The Convergence of BSUM [Razaviyayn-H.-Luo 13]

- Cyclic coordinate selection rule
- Assumption A holds, and that each x^r is **regular**
- Suppose $X^0 = \{x \mid f(x) \leq f(x^0)\}$ is a compact set
- The BSUM subproblem has a **unique** solution for any point $x^{r-1} \in \mathcal{X}$.

Then every limit point x^* of $\{x^r\}$ is a stationary point.

“A Block Successive Upper Bound Minimization Algorithm for Nonsmooth Nonconvex Optimization”, *SIAM Journal on Optimization*, 2013.

Main Convergence Analysis

- Works for a large family of approximation functions
- Uniqueness of subproblems easy to achieve – choose an appropriate upper bound
- Regularity has to be verified beforehand
- Convergence to **stationary point** – almost the best one can hope for considering the nonconvexity involved
- Result can be significantly improved for **convex problems**

How Fast Does BSUM Converge?

- Let us characterize the convergence speed of the algorithm
- Important issue, especially for big data problems
- Extensive work on analyzing special cases of BSUM [Nesterov 12] [Richtárik et al 14] [Lu and Xiao 13] [Beck and Tetruashvili 13]
- Here we present a general rate analysis for BSUM
- We will discuss this result further in the afternoon

The Convergence Rate Analysis (Convex Case)

• Main Assumptions

(B1) $f(x) := g(x) + \sum_{i=1}^n h_i(x_i)$, where $g(x)$ is a **smooth convex** function with **Lipschitz continuous gradient**, i.e. there exists a constant L such that

$$\|\nabla g(x) - \nabla g(y)\| \leq L\|x - y\|, \forall x, y \in X.$$

Further **both g and h_i 's are convex functions.**

(B2) The level set $\{x \mid f(x) \leq f(x^0), x \in X\}$ is compact.

The Convergence Rate Analysis: Main Result

- **Convex case** [H. -Wang-Razaviyayn-Luo 13]

Define an **ϵ -optimal solution** as

$$x^\epsilon := \{x \mid x \in X, f(x) - f(x^*) \leq \epsilon\}$$

Under Assumption A - B, BSUM takes at most c/ϵ iterations to find an ϵ -optimal solution (**sublinear convergence**)

“Iteration Complexity Analysis for Block Coordinate Descent Type Methods”, *Math Programming*, minor revision, 2015.

Implications and Generalizations

- Our rate analysis generalizes to many coordinates selection rules: randomized, Gauss-Southwell, Essentially cyclic, etc.
- The traditional BCD method (as a special case) converges with the same rate for convex problems
 - *Example:* The well-known **iterative water-filling algorithm (IWFA)** for computing MIMO MAC channel capacity [Yu *et al* 04] converges sublinearly
- In the absence of convexity, one can show that the algorithm converges with a rate $\mathcal{O}(1/\epsilon^2)$ [Razaviyayn-H.-Luo-Pang 14]

“Parallel successive convex approximation for nonsmooth nonconvex optimization”, *NIPS*, 2014.

Outline

- 1 Introduction: Block Structured Problems
- 2 The BSUM Framework
- 3 Properties of BSUM
- 4 Algorithms Covered By BSUM**
- 5 Applications of BSUM in SP, ML, COM
- 6 Extension
- 7 Conclusion

Main Theme

- We will introduce a few applications of BSUM
 - Classical algorithms that are BSUM in disguise
 - New applications in ML, SP, COM...
- **Key:** Properly identify the upper-bounds
 - Simplify computation
 - Decompose the problem

Algorithm 1: CCCP

- Consider the following difference of convex (DC) program:

$$\min f(x) := g_1(x) - g_2(x),$$

where both g_1 and g_2 are **convex** functions

- The well-known CCCP algorithm [Yuille *et al* 2003] generates a sequence $\{x^r\}$ by solving

$$x^{r+1} = \arg \min_x u(x, x^r),$$

where $u(x, x^r) := g_1(x) - (\langle x - x^r, \nabla g_2(x^r) \rangle + g_2(x^r))$.

- Clearly, the **linear** upper-bound is used
- A **single block** version of BSUM with **smooth** objective

Algorithm 1: CCCP

- Consider the following difference of convex (DC) program:

$$\min f(x) := g_1(x) - g_2(x),$$

where both g_1 and g_2 are **convex** functions

- The well-known CCCP algorithm [Yuille *et al* 2003] generates a sequence $\{x^r\}$ by solving

$$x^{r+1} = \arg \min_x u(x, x^r),$$

where $u(x, x^r) := g_1(x) - (\langle x - x^r, \nabla g_2(x^r) \rangle + g_2(x^r))$.

- Clearly, the **linear** upper-bound is used
- A **single block** version of BSUM with **smooth** objective

Algorithm 1: CCCP

- Consider the following difference of convex (DC) program:

$$\min f(x) := g_1(x) - g_2(x),$$

where both g_1 and g_2 are **convex** functions

- The well-known CCCP algorithm [Yuille *et al* 2003] generates a sequence $\{x^r\}$ by solving

$$x^{r+1} = \arg \min_x u(x, x^r),$$

where $u(x, x^r) := g_1(x) - (\langle x - x^r, \nabla g_2(x^r) \rangle + g_2(x^r))$.

- Clearly, the **linear** upper-bound is used
- A **single block** version of BSUM with **smooth** objective

Algorithm 1: CCCP

- Consider the following difference of convex (DC) program:

$$\min f(x) := g_1(x) - g_2(x),$$

where both g_1 and g_2 are **convex** functions

- The well-known CCCP algorithm [Yuille *et al* 2003] generates a sequence $\{x^r\}$ by solving

$$x^{r+1} = \arg \min_x u(x, x^r),$$

where $u(x, x^r) := g_1(x) - (\langle x - x^r, \nabla g_2(x^r) \rangle + g_2(x^r))$.

- Clearly, the **linear** upper-bound is used
- A **single block** version of BSUM with **smooth** objective

Algorithm 2: Majorization-Minimization (MM) Algorithm

- The MM has been widely used in statistics [Hunter 00, 04]
- Consider the problem of $\min_{x \in X} f(x)$ where $f(x)$ is smooth
- MM first constructs a “majorization” function $u(x, z)$ for the original objective $f(z)$, such that

$$u(x, z) \geq f(z), \forall x, z \in X, \quad u(x, x) = f(x), \forall x \in X.$$

- Then the algorithm generates a sequence of iterates by successively minimizing $u(x, x^r)$
- A single block version of BSUM with smooth objective

Algorithm 2: Majorization-Minimization (MM) Algorithm

- The MM has been widely used in statistics [Hunter 00, 04]
- Consider the problem of $\min_{x \in X} f(x)$ where $f(x)$ is smooth
- MM first constructs a “majorization” function $u(x, z)$ for the original objective $f(z)$, such that

$$u(x, z) \geq f(z), \forall x, z \in X, \quad u(x, x) = f(x), \forall x \in X.$$

- Then the algorithm generates a sequence of iterates by successively minimizing $u(x, x^r)$
- A single block version of BSUM with smooth objective

Algorithm 2: Majorization-Minimization (MM) Algorithm

- The MM has been widely used in statistics [Hunter 00, 04]
- Consider the problem of $\min_{x \in X} f(x)$ where $f(x)$ is smooth
- MM first constructs a “majorization” function $u(x, z)$ for the original objective $f(z)$, such that

$$u(x, z) \geq f(z), \forall x, z \in X, \quad u(x, x) = f(x), \forall x \in X.$$

- Then the algorithm generates a sequence of iterates by successively minimizing $u(x, x^r)$
- A single block version of BSUM with smooth objective

Algorithm 2: Majorization-Minimization (MM) Algorithm

- The MM has been widely used in statistics [Hunter 00, 04]
- Consider the problem of $\min_{x \in X} f(x)$ where $f(x)$ is smooth
- MM first constructs a “majorization” function $u(x, z)$ for the original objective $f(z)$, such that

$$u(x, z) \geq f(z), \forall x, z \in X, \quad u(x, x) = f(x), \forall x \in X.$$

- Then the algorithm generates a sequence of iterates by successively minimizing $u(x, x^r)$
- A single block version of BSUM with smooth objective

Algorithm 2: Majorization-Minimization (MM) Algorithm

- The MM has been widely used in statistics [Hunter 00, 04]
- Consider the problem of $\min_{x \in X} f(x)$ where $f(x)$ is smooth
- MM first constructs a “majorization” function $u(x, z)$ for the original objective $f(z)$, such that

$$u(x, z) \geq f(z), \forall x, z \in X, \quad u(x, x) = f(x), \forall x \in X.$$

- Then the algorithm generates a sequence of iterates by successively minimizing $u(x, x^r)$
- A single block version of BSUM with smooth objective

Special Case: the EM algorithm

- Consider the celebrated EM algorithm [Dempster 77]
- Let w be a vector observation; x **unknown parameter**
- The maximum likelihood estimate of x is given as

$$\hat{x}_{\text{ML}} = \arg \max_x \ln p(w|x).$$

- Let the random vector z be some hidden/unobserved variable
- The EM generates a sequence $\{x^r\}$ by iteratively updating
 - E-Step: Calculate $g(x, x^r) := \mathbb{E}_{z|w, x^r} \{\ln p(w, z|x)\}$;
 - M-Step: $x^{r+1} = \arg \max_x g(x, x^r)$.

Special Case: The EM Algorithm

- Steps of derivation

$$\begin{aligned}
 -\ln p(w|x) &= -\ln \mathbb{E}_{z|x} p(w|z, x) \\
 &= -\ln \mathbb{E}_{z|x} \left[\frac{p(z|w, x^r) p(w|z, x)}{p(z|w, x^r)} \right] \\
 &= -\ln \mathbb{E}_{z|w, x^r} \left[\frac{p(z|x) p(w|z, x)}{p(z|w, x^r)} \right] \\
 &\leq -\mathbb{E}_{z|w, x^r} \ln \left[\frac{p(z|x) p(w|z, x)}{p(z|w, x^r)} \right] \\
 &= \underbrace{-\mathbb{E}_{z|w, x^r} \ln p(w, z|x)}_{\text{the bound}} + \underbrace{\mathbb{E}_{z|w, x^r} \ln p(z|w, x^r)}_{\text{not a function of } x} \\
 &:= u(x, x^r),
 \end{aligned}$$

- Jensen's inequality is used to derive the upper-bound

- Multiblock EM?

Special Case: The EM Algorithm

- Steps of derivation

$$\begin{aligned}
 -\ln p(w|x) &= -\ln \mathbb{E}_{z|x} p(w|z, x) \\
 &= -\ln \mathbb{E}_{z|x} \left[\frac{p(z|w, x^r) p(w|z, x)}{p(z|w, x^r)} \right] \\
 &= -\ln \mathbb{E}_{z|w, x^r} \left[\frac{p(z|x) p(w|z, x)}{p(z|w, x^r)} \right] \\
 &\leq -\mathbb{E}_{z|w, x^r} \ln \left[\frac{p(z|x) p(w|z, x)}{p(z|w, x^r)} \right] \\
 &= \underbrace{-\mathbb{E}_{z|w, x^r} \ln p(w, z|x)}_{\text{the bound}} + \underbrace{\mathbb{E}_{z|w, x^r} \ln p(z|w, x^r)}_{\text{not a function of } x} \\
 &:= u(x, x^r),
 \end{aligned}$$

- Jensen's inequality is used to derive the upper-bound
- Multiblock EM?

Algorithm 3: The Forward-Backward Splitting (FBS)

- The FBS [Combettes 09] is popular for nonsmooth optimization (i.e., compressive sensing); Consider

$$\min_{x \in \mathcal{X}} f(x) := g(x) + h(x)$$

h is convex nonsmooth; g convex, has Lipschitz gradient with constant L

- Define the proximity operator $\text{prox}_h : \mathcal{X} \rightarrow \mathcal{X}$ as

$$\text{prox}_h(x) = \arg \min_{y \in \mathcal{X}} h(y) + \frac{1}{2} \|x - y\|^2.$$

Algorithm 3: The Forward-Backward Splitting (FBS)

- The FBS [Combettes 09] is popular for nonsmooth optimization (i.e., compressive sensing); Consider

$$\min_{x \in \mathcal{X}} f(x) := g(x) + h(x)$$

h is convex nonsmooth; g convex, has Lipschitz gradient with constant L

- Define the **proximity operator** $\text{prox}_h : \mathcal{X} \rightarrow \mathcal{X}$ as

$$\text{prox}_h(x) = \arg \min_{y \in \mathcal{X}} h(y) + \frac{1}{2} \|x - y\|^2.$$

Algorithm 3: The Forward-Backward Splitting (FBS)

- The FBS iteration

$$x^{r+1} = \underbrace{\text{prox}_{\beta h}}_{\text{backward step}} \left(\underbrace{x^r - \beta \nabla g(x^r)}_{\text{forward step}} \right)$$

with $\beta \in [\epsilon, 2/L - \epsilon]$, $\epsilon \in]0, \min\{1, 1/L\}[$

- Equivalently it is successive optimizing the following bound

$$u(x, x^r) := h(x) + \frac{1}{2\beta} \|x - x^r\|^2 + \langle x - x^r, \nabla g(x^r) \rangle + g(x^r).$$

quadratic upper bound of $g(x)$

- Multi-block FBS?

Algorithm 3: The Forward-Backward Splitting (FBS)

- The FBS iteration

$$x^{r+1} = \underbrace{\text{prox}_{\beta h}}_{\text{backward step}} \left(\underbrace{x^r - \beta \nabla g(x^r)}_{\text{forward step}} \right)$$

with $\beta \in [\epsilon, 2/L - \epsilon]$, $\epsilon \in]0, \min\{1, 1/L\}[$

- Equivalently it is successive optimizing the following bound

$$u(x, x^r) := h(x) + \frac{1}{2\beta} \|x - x^r\|^2 + \langle x - x^r, \nabla g(x^r) \rangle + g(x^r).$$

quadratic upper bound of $g(x)$

- Multi-block FBS?

Algorithm 3: The Forward-Backward Splitting (FBS)

- The FBS iteration

$$x^{r+1} = \underbrace{\text{prox}_{\beta h}}_{\text{backward step}} \left(\underbrace{x^r - \beta \nabla g(x^r)}_{\text{forward step}} \right)$$

with $\beta \in [\epsilon, 2/L - \epsilon]$, $\epsilon \in]0, \min\{1, 1/L\}[$

- Equivalently it is successive optimizing the following bound

$$u(x, x^r) := h(x) + \frac{1}{2\beta} \|x - x^r\|^2 + \langle x - x^r, \nabla g(x^r) \rangle + g(x^r).$$

quadratic upper bound of $g(x)$

- Multi-block FBS?

Algorithm 4: Multiplicative NMF (M-NMF)

- Consider the following NMF problem:

$$\min_{W \in \mathbb{R}^{M \times K}, H \in \mathbb{R}^{K \times M}} f(W, H) := \frac{1}{2} \|V - WH\|_F^2, \quad \text{s.t. } W \geq 0, H \geq 0,$$

where $V \in \mathbb{R}_+^{M \times N}$ is given.

- Wide application including factor analysis, dictionary learning, speech analysis and so on [Gillis 2014]
- A well-known multiplicative algorithm [Lee-Seung 00]

$$[H^{r+1}]_{j,i} = [H^r]_{j,i} \frac{[(W^r)^T V]_{j,i}}{[(W^r)^T W^r H^r]_{j,i}}, \forall j, i$$

$$[W^{r+1}]_{j,i} = [W^r]_{j,i} \frac{[V(H^{r+1})^T]_{j,i}}{[W^r H^{r+1} (H^{r+1})^T]_{j,i}}, \forall j, i.$$

Here $[H^{r+1}]_{j,i}$ means the (j, i) th component of matrix H^{r+1} .

Algorithm 4: Multiplicative NMF (M-NMF)

- Consider the following NMF problem:

$$\min_{W \in \mathbb{R}^{M \times K}, H \in \mathbb{R}^{K \times M}} f(W, H) := \frac{1}{2} \|V - WH\|_F^2, \quad \text{s.t. } W \geq 0, H \geq 0,$$

where $V \in \mathbb{R}_+^{M \times N}$ is given.

- Wide application including factor analysis, dictionary learning, speech analysis and so on [Gillis 2014]
- A well-known multiplicative algorithm [Lee-Seung 00]

$$[H^{r+1}]_{j,i} = [H^r]_{j,i} \frac{[(W^r)^T V]_{j,i}}{[(W^r)^T W^r H^r]_{j,i}}, \forall j, i$$

$$[W^{r+1}]_{j,i} = [W^r]_{j,i} \frac{[V(H^{r+1})^T]_{j,i}}{[W^r H^{r+1} (H^{r+1})^T]_{j,i}}, \forall j, i.$$

Here $[H^{r+1}]_{j,i}$ means the (j, i) th component of matrix H^{r+1} .

Algorithm 4: Multiplicative NMF (M-NMF)

- Consider the following NMF problem:

$$\min_{W \in \mathbb{R}^{M \times K}, H \in \mathbb{R}^{K \times M}} f(W, H) := \frac{1}{2} \|V - WH\|_F^2, \quad \text{s.t. } W \geq 0, H \geq 0,$$

where $V \in \mathbb{R}_+^{M \times N}$ is given.

- Wide application including factor analysis, dictionary learning, speech analysis and so on [Gillis 2014]
- A well-known multiplicative algorithm [Lee-Seung 00]

$$[H^{r+1}]_{j,i} = [H^r]_{j,i} \frac{[(W^r)^T V]_{j,i}}{[(W^r)^T W^r H^r]_{j,i}}, \forall j, i$$

$$[W^{r+1}]_{j,i} = [W^r]_{j,i} \frac{[V(H^{r+1})^T]_{j,i}}{[W^r H^{r+1} (H^{r+1})^T]_{j,i}}, \forall j, i.$$

Here $[H^{r+1}]_{j,i}$ means the (j, i) th component of matrix H^{r+1} .

Algorithm 4: multiplicative NMF (M-NMF)

- The M-NMF is covered by BSUM (with **2 blocks**)
- Let H_i and V_i represent the i th column of H and V
- At a given iterate $\{W^r, H^r\}$, optimize H_i

$$\min_{H_i \geq 0} f(H_i, \{W^r, H^r\}) := \frac{1}{2} \|V_i - W^r H_i\|_F^2 + \frac{1}{2} \sum_{j \neq i} \|V_j - W^r H_j^r\|_F^2.$$

- Define the **upper bound** function as

$$\begin{aligned} & f(H_i^r, \{W^r, H^r\}) + (H_i - H_i^r)^T \nabla_{H_i} f(H_i, \{W^r, H^r\}) \\ & + \frac{1}{2} (H_i - H_i^r)^T \Phi_i(W^r, H_i^r) (H_i - H_i^r) \end{aligned}$$

where $\Phi_i(W^r, H_i^r)$ is a diagonal matrix given by

$$\Phi_i(W^r, H_i^r) := \text{Diag} \left(\frac{[(W^r)^T W^r H_i^r]_1}{[H_i^r]_1}, \dots, \frac{[(W^r)^T W^r H_i^r]_K}{[H_i^r]_K} \right)$$

- Minimizing the bound exactly yields the H-step

Applications

- The algorithms we see so far are classic/generic
- Many new applications of BSUM
 - Speech denoising [Tseng *et al* 13]
 - Distributed optimizing Multi-Agent system [Scutari *et al* 12]
 - RNA sequencing [Bazerque *et al* 12]
 - Wireless Information and Energy Transfer [Shen *et al* 13]
 - Visual rendering of graphs [Biangana *et al* 12]

Outline

- 1 Introduction: Block Structured Problems
- 2 The BSUM Framework
- 3 Properties of BSUM
- 4 Algorithms Covered By BSUM
- 5 Applications of BSUM in SP, ML, COM**
- 6 Extension
- 7 Conclusion

Application in Transceiver Design

- Consider a multi-input multi-output interference channel with K transmitter-receiver pairs
- Let M (resp. N) be the number of antennas at each transmitter (resp. receiver) and each transmitter k
- Let $\mathbf{x}_k \in \mathbb{C}^M$ be the transmitted signal of user k
- The received signal of user k is given by

$$\mathbf{y}_k = \underbrace{\mathbf{H}_{kk}\mathbf{x}_k}_{\text{desired signal}} + \underbrace{\sum_{j \neq k} \mathbf{H}_{kj}\mathbf{x}_j}_{\text{multi-user interference}} + \underbrace{\mathbf{n}_k}_{\text{noise}}$$

where $\mathbf{H}_{kj} \in \mathbb{C}^{N \times M}$ is the channel from transmitter j to receiver k

Application in Transceiver Design

- When linear beamformers are used, the transmitted signal and the estimated received data is given by

$$\mathbf{x}_k = \mathbf{v}_k s_k \quad \text{and} \quad \hat{s}_k = \mathbf{u}_k^H \mathbf{y}_k,$$

- $\mathbf{v}_k$ and $\mathbf{u}_k$ are respectively the transmit and receive beamformers
- Consider the following sum rate utility maximization problem

$$\max_{\mathbf{u}, \mathbf{v}} \sum_{k=1}^K R_k(\mathbf{u}, \mathbf{v}), \quad \text{s.t.} \quad \|\mathbf{v}_k\|^2 \leq P_k, \quad \forall k.$$

- $R_k(\mathbf{u}, \mathbf{v})$, the rate of user k , is given by

$$R_k(\mathbf{u}, \mathbf{v}) = \log \left(1 + \frac{|\mathbf{u}_k^H \mathbf{H}_{kk} \mathbf{v}_k|^2}{\sigma^2 \|\mathbf{u}_k\|^2 + \sum_{j \neq k} |\mathbf{u}_k^H \mathbf{H}_{kj} \mathbf{v}_j|^2} \right).$$

Application in Transceiver Design

- Using the relation between SINR/MSE, rewrite it as

$$\begin{aligned} \min_{\mathbf{v}, \mathbf{u}} \quad & \sum_{k=1}^K \log(e_k(\mathbf{u}, \mathbf{v})) \\ \text{s.t.} \quad & \|\mathbf{v}_k\|^2 \leq P_k, \quad \forall k = 1, 2, \dots, K, \end{aligned}$$

where $e_k(\mathbf{u}, \mathbf{v})$ is the MSE value and is given by

$$e_k(\mathbf{u}, \mathbf{v}) = |\mathbf{u}_k^H \mathbf{H}_{kk} \mathbf{v}_k - 1|^2 + \sum_{j \neq k} |\mathbf{u}_k^H \mathbf{H}_{kj} \mathbf{v}_j|^2 + \sigma^2.$$

Application in Transceiver Design

- Since the $\log(\cdot)$ function is **concave**, use linear upper-bound

$$u(\mathbf{x}, \mathbf{x}^r) = \sum_{k=1}^K \left(\log(e_k(\mathbf{x}^r)) + (e_k(\mathbf{x}^r))^{-1} (e_k(\mathbf{x}) - e_k(\mathbf{x}^r)) \right)$$

where $\mathbf{x} \triangleq (\mathbf{u}, \mathbf{v})$ is the optimization variable

- This choice of approximation function leads to a **quadratic programming** problem which has closed form solutions
- Further, fixing $\mathbf{v}$ (resp. $\mathbf{u}$), the subproblems are **completely decoupled** among the users
- Leads to parallel implementation – well known WMMSE algorithm [Shi *et al* 2011] and its extensions [H.-Sun-Luo 2012][Baligh *et al* 2014]

Applications: tensor decomposition

- Tensor $\mathfrak{X} \in \mathbb{R}^{I \times J \times K}$
- CANDECOMP/PARAFAC decomposition:

$$\mathfrak{X} = \sum_{\ell=1}^L a_{\ell} \circ b_{\ell} \circ c_{\ell}$$

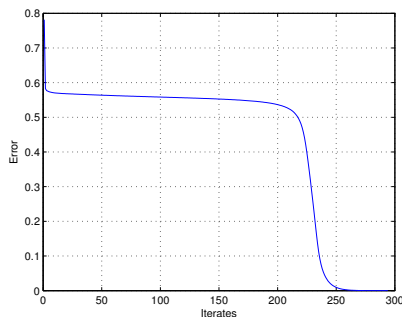
- a_{ℓ} : ℓ -th column of A ; similarly define b_{ℓ}, c_{ℓ}
- $A \in \mathbb{R}^{I \times L}$, $B \in \mathbb{R}^{J \times L}$, and $C \in \mathbb{R}^{K \times L}$
- NP-hard [Hastad 90]
- Alternating Least Squares (ALS) [Carroll *et al.* 70] [Harshman 70]

$$\min_{A,B,C} \left\| \mathfrak{X} - \sum_{\ell=1}^L a_{\ell} \circ b_{\ell} \circ c_{\ell} \right\|^2$$

- Use BCD, closed form updates

Drawbacks of ALS algorithm

- Convergence is not guaranteed (No unique minimizer)
 - [Chen *et al.*'12]: Maximum block improvement, super-symmetric rank one tensors
 - [Uschmajew'10]: ALS with re-scaling, *local* linear convergence under the positive definiteness assumption of Hessian
- Swamp effect



ALS with diminishing proximal term

- [Navasca *et al.* 2008]: Adding **proximal term** can help reducing swamp

$$\min_{A,B,C} \left\| \mathfrak{X} - \sum_{r=1}^R a_r \circ b_r \circ c_r \right\|^2 + \lambda^r (\|A - A^r\|^2 + \|B - B^r\|^2 + \|C - C^r\|^2)$$

- λ^r is decreasing, e.g.,

$$\lambda^r = \lambda_0 + \lambda_1 \frac{\left\| \mathfrak{X} - \sum_{\ell=1}^L a_\ell \circ b_\ell \circ c_\ell \right\|}{\|\mathfrak{X}\|}$$

- Still closed form updates
- Convergence is not known,, till now.
- BSUM convergence $\Rightarrow$ convergence of ALS with diminishing proximal term
- How does it perform in practice?

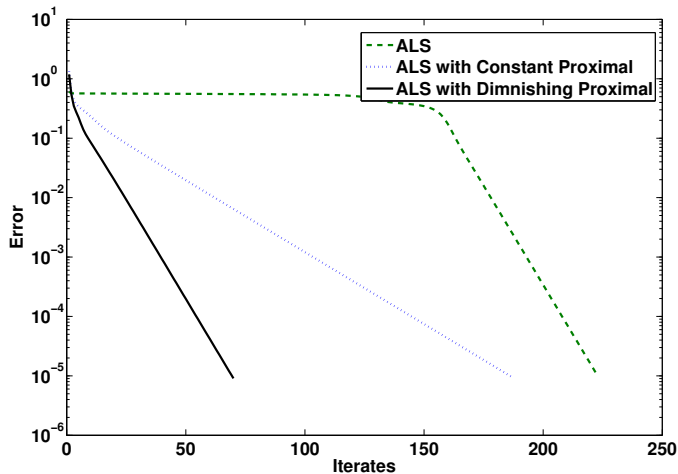
Tensor decomposition - simulations

- $\mathfrak{X}$ is obtained from the decomposition (Example from [Navasca *et al.* 2008])

$$A = \begin{bmatrix} 1 & \cos \theta & 0 \\ 0 & \sin \theta & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 3 & \sqrt{2} \cos \theta & 0 \\ 0 & \sin \theta & 1 \\ 0 & \sin \theta & 0 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Random initialization
- Different methods
 - No proximal term
 - Constant proximal coefficient
 - Diminishing proximal coefficient
 - Gauss-Southwell/Maximum Block Improvement
 - Maximum Improvement Successive Upper-bound Minimization (MISUM) using diminishing proximal coefficient

Tensor decomposition - simulations



Tensor decomposition - simulations

- Averaged over 1000 Monte-Carlo runs
- Termination rule: error $\leq 10^{-5}$

Algorithm	Number of iterations to converge
ALS	277
Constant Proximal	140
Diminishing Proximal	78
MBI	572
MISUM	175

Outline

- 1 Introduction: Block Structured Problems
- 2 The BSUM Framework
- 3 Properties of BSUM
- 4 Algorithms Covered By BSUM
- 5 Applications of BSUM in SP, ML, COM
- 6 Extension**
- 7 Conclusion

Stochastic Optimization

- The BSUM framework can be extended to solve the following problem [Razaviyayn *et al* 14]

$$\begin{aligned} \min_x \quad & f(x) \triangleq \mathbb{E}_{\xi} [g(x, \xi)] \\ \text{s.t.} \quad & x \in \mathcal{X}, \end{aligned}$$

- ϵ is a given random variable modeling uncertainty in the problem
- Applications in online dictionary learning, online transceiver design

Online Transceiver Design

- Channel coefficients $\{\mathbf{H}_{ij}\}_{i,j}$ are not exactly known
- Consider [Razaviyayn-Sanjabi-Luo 13]

$$\begin{aligned} \max_{\mathbf{u}, \mathbf{v}} \quad & \sum_{k=1}^K \mathbb{E}_{\mathbf{H}} \left[R_k(\mathbf{u}, \mathbf{v}) \right] \\ \text{s.t.} \quad & \|\mathbf{v}_k\|^2 \leq P_k, \quad \forall k = 1, 2, \dots, K, \end{aligned} \quad (6.5)$$

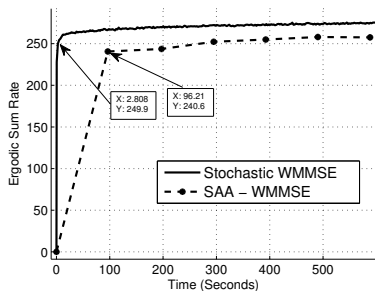
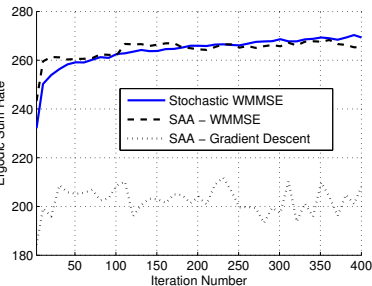


Figure: Iteration and running time comparison of SSIM vs. SAA

Linearly Coupling Constraints (convex case)

- Consider a convex problem with linear coupling constraints

$$\begin{aligned} \min_{(x_1, \dots, x_n)} \quad & f(x_1, \dots, x_n) \\ \text{s.t.} \quad & \sum_{i=1}^n A_i x_i = b, \quad x_i \in \mathcal{X}_i, \quad \forall i \end{aligned}$$

- Can be solved by combining BSUM with the Alternating Direction Method of Multipliers [H.- et al 14]
- Will discuss something along this line in the subsequent lectures

Linearly Coupling Constraints (nonconvex case)

- Consider a **nonconvex** problem with linear coupling constraints

$$\begin{aligned} \min_{(x_1, \dots, x_n)} \quad & f(x_1, \dots, x_n) \\ \text{s.t.} \quad & \sum_{i=1}^n A_i x_i = b, \quad x_i \in \mathcal{X}_i, \quad \forall i \end{aligned}$$

- Can be solved by combining BSUM with certain primal- dual method [H. 16] [H.-Luo-Razaviyayn 15]
- Will discuss something along this line in the subsequent lectures

H., “Decomposing Nonconvex Problems using a Proximal Primal-Dual Approach: Algorithms, Convergence, and Applications”, preprint, 2016.
 H., Z.-Q. Luo and M. Razaviyayn, “Convergence Analysis of ADMM for a family of nonconvex problems”, to appear, *SIAM Journal on Optimization*, 2015.

Linearly Coupling Constraints (nonconvex case)

- Application in **nonconvex** distributed network optimization
- Suppose N nodes in the network, each one possesses a **local** function f_i ; they jointly solve the following problem

$$\min_{y \in \mathbb{R}^N} \sum_{i=1}^N f_i(y) \quad (6.6)$$

- This problem can be equivalently modeled as

$$\min_{\{x_i\}} \sum_{i=1}^N f_i(x_i), \quad \text{s.t. } Ax = 0 \quad (6.7)$$

where A is the incident matrix of the graph

- Using a variant of BSUM + Primal Dual method, recovers a convergent distributed optimization algorithm [H. 16]

Multi-Commodity Routing Problem (TE)

- Max-min fairness problem (a linear program)

$$\max_{\mathbf{f}, \mathbf{r}} \quad r_{\min} \quad \text{s.t.} \quad \mathbf{f} \geq 0, \quad r_m \geq r_{\min}, \quad m = 1, \dots, M \quad (6.8a)$$

$$(1.1) \text{ and } (1.2), \quad (6.8b)$$

- Apply the BSUM extension to linearly coupling problem [Liao-Hong-Farmanbar-Luo 14]
- Problem completely decouples among the NCs, parallel implementation

“Min flow rate maximization for software defined radio access networks”, *JSAC*, 2014

Multi-Commodity Routing Problem (TE)

- A network with 126 nodes partitioned into 9 subnets
- A total of 306 links
- The capacities for the links within (resp. between) the subnetworks are uniformly distributed in [50,100] MBits/s (resp. [20,50] MBits/s)
- Use a HPC cluster; make each computing node to be a NC

Table: Comparison with Different Algorithms. $M = 200$

Approaches	Time	# of iterations
Gurobi	11.4690s	N/A
BSUMM (10 NCs, 1 core)	18.0679s	264.71
BSUMM (10 NCs, 10 core)	4.23s	2109.07

Parallelization

- A variants of BSUM allows the blocks to be updated **in parallel** – suitable for HPC implementation [Razaviyayn-H.-Luo-Pang 14]
- **Key idea**: Update multiple blocks at the same time + a proper stepsize selection

- et $x_i^r = x_i^{r-1} + \gamma^r \times \underbrace{(\hat{x}_i^r - x_i^{r-1})}_{\text{A direction computed by BSUM}}, \quad \forall i \in \mathcal{I}^r$

A direction computed by BSUM

“Parallel successive convex approximation for nonsmooth nonconvex optimization”, *NIPS*, 2014.

Parallelization

- 1 Find $x^0 \in \mathcal{X}$; set $r = 0$, and choose a stepsize sequence $\{\gamma^r\}$
- 2 **repeat**
- 3 Pick index set $\mathcal{I}^r$
- 3 Let $\mathcal{Z}_i^r = \arg \min_{x_i \in \mathcal{X}_i} u_i(x_i, x^{r-1}), \forall i \in \mathcal{I}^r$
- 4 Set $\hat{x}_i^r$ to be an arbitrary element in $\mathcal{Z}_i^r, \forall i \in \mathcal{I}^r$
- 5 Set $x_k^r = x_k^{r-1}, \forall k \notin \mathcal{I}^r$
- 6 Set $x_i^r = x_i^{r-1} + \gamma^r(\hat{x}_i^r - x_i^{r-1}), \forall i \in \mathcal{I}^r$
- 7 $r = r + 1,$
- 8 **until** some convergence criterion is met

Table: Pseudo code of the PSCA algorithm

Parallelization

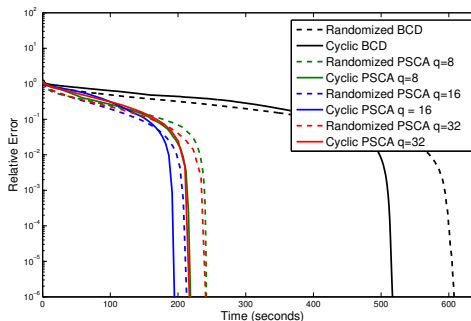
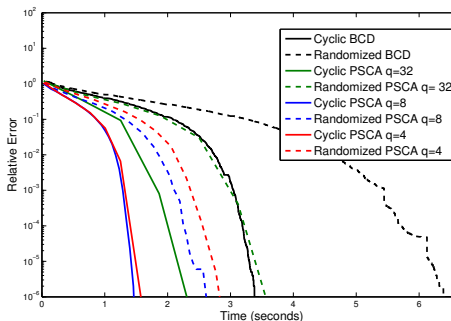


Figure: Comparison of the serial BCD with PSCA method for solving the Lasso problem: $\min_x \|Ax - b\|^2 + \lambda \|x\|_1$. The dimension of A is $2,000 \times 10,000$ for the figure in left and $1,000 \times 100,000$ for the one in right.

Outline

- 1 Introduction: Block Structured Problems
- 2 The BSUM Framework
- 3 Properties of BSUM
- 4 Algorithms Covered By BSUM
- 5 Applications of BSUM in SP, ML, COM
- 6 Extension
- 7 Conclusion**

Conclusion

- We have introduced a framework for optimizing block-structured optimization problems
- Generic convergence/rate of convergence analysis
- Covers lots of known algorithms as special cases (with possible generalizations)
- Many applications in SP, ML, COM
- Active research areas, many open problems

Thank You! Questions?